Interfaces et Modularité

---

1. Les Interfaces en Go : Polymorphisme par Satisfaction Implicite

En Java, C# ou C++, pour qu'une classe implémente une interface, le développeur doit le déclarer explicitement au moment où il écrit la classe (`class PostgresDB implements Database`). Cela crée un couplage fort vers l'amont : si vous utilisez une bibliothèque tierce qui ne déclare pas votre interface, vous devez écrire un *Wrapper/Adapter*.

En Go, **le mot-clé `implements` n'existe pas.** Les interfaces sont satisfaites de manière **100 % implicite** (*Structural Typing* ou *Duck Typing* vérifié statiquement à la compilation).

1.1 Définition et mécanisme

Une interface définit un **comportement** (une liste de signatures de méthodes). N'importe quel type (de votre code ou d'une librairie externe) qui possède l'ensemble des méthodes listées par l'interface **est** automatiquement considéré par le compilateur comme implémentant cette interface.

```
Go
package main

import "fmt"

// 1. Définition du contrat (1 seule méthode : idiome très fréquent en Go)
type Notificateur interface {
	Envoyer(message string) bool
}

// 2. Implémentation A : Email (Aucune mention de 'Notificateur' !)
type ClientEmail struct {
	ServeurSMTP string
}

func (e ClientEmail) Envoyer(message string) bool {
	fmt.Printf("[SMTP %s] Email envoyé : %s\n", e.ServeurSMTP, message)
	return true
}

// 3. Implémentation B : SMS (Aucune mention de 'Notificateur' !)
```

```go
type ClientSMS struct {
	NumeroTelephone string
}

func (s *ClientSMS) Envoyer(message string) bool {
	fmt.Printf("[SMS %s] Texte envoyé : %s\n", s.NumeroTelephone, message)
	return true
}

// 4. Fonction polymorphe : accepte n'importe quel type satisfaisant
'Notificateur'
func AlerterAdministrateur(n Notificateur, alerte string) {
	n.Envoyer("URGENT : " + alerte)
}

func main() {
	email := ClientEmail{ServeurSMTP: "smtp.interne.local"}
	sms := &ClientSMS{NumeroTelephone: "+33612345678"}

	AlerterAdministrateur(email, "Disque plein à 95%")
	AlerterAdministrateur(sms, "Disque plein à 95%")
}
```

1.2 Règle critique : *Method Sets* (Value Receiver vs Pointer Receiver)

Pourquoi avons-nous passé `email` par valeur (`ClientEmail`), mais `sms` par pointeur (`&ClientSMS`) dans l'exemple ci-dessus ?

À cause de la règle des **Ensembles de méthodes (*Method Sets*)** du compilateur Go :

- Si une méthode est définie sur un **Value Receiver** `(t T)`, alors le type valeur `T` **ET** le type pointeur `*T` satisfont tous les deux l'interface.
- Si une méthode est définie sur un **Pointer Receiver** `(t *T)`, alors **SEUL le pointeur `*T`** satisfait l'interface ! Passer la valeur `T` dans l'interface est refusé à la compilation (`ClientSMS does not implement Notificateur (Envoyer method has pointer receiver)`), car une valeur stockée dans une interface n'est pas adressable de manière sécurisée.

1.3 Les règles d'or d'architecture en Go

1. **Des interfaces minuscules (1 à 2 méthodes max) :** En Go, par convention, une interface d'une seule méthode porte le nom du verbe terminé par `-er` (`io.Reader`, `io.Writer`, `fmt.Stringer`, `io.Closer`). On peut ensuite composer des interfaces entre elles :

```go
type ReadWriter interface {
    Reader // Interface Embedding !
    Writer
}
```

2. **Définir l'interface du côté du consommateur (et non du producteur) :** Ne créez pas un package `database` contenant à la fois la `struct Postgres` et une énorme interface `IDatabase` de 20 méthodes (réflexe Java/C#). Créez uniquement la `struct` concrète, et laissez le package appelant définir sa propre petite interface de 2 méthodes correspondant exactement à son besoin (facilite grandement le *Mocking* lors des tests unitaires).
3. **Adage officiel : *"Accept interfaces, return structs"* :** Vos fonctions doivent accepter des interfaces en paramètres (flexibilité maximale en entrée) et retourner des types concrets `struct` (clarté maximale en sortie).

---

2. Sous le capot d'une Interface (`iface`) et le piège du `nil`

Pour un développeur C, comprendre comment Go représente une interface en mémoire permet d'éviter le bug le plus célèbre de Go en production.

2.1 Anatomie mémoire : Le couple `(tab, data)` de 16 octets

Une variable de type interface en Go n'est **pas** un simple pointeur de 8 octets. C'est une structure interne du runtime de **16 octets** (`iface`, ou `eface` pour l'interface vide `any`) composée de **deux pointeurs de 8 octets** :

1. **`tab` (`*itab` / Type) :** Un pointeur vers une table décrivant le **type concret dynamique** stocké et la table de ses pointeurs de fonctions (équivalent de la *vtable* en C++).
2. **`data` (`unsafe.Pointer` / Valeur) :** Un pointeur vers la **donnée concrète** allouée en mémoire.

2.2 Le piège mortel du pointeur typé `nil` dans une interface !

Une interface est considérée comme égale à `nil` (`monInterface == nil`) **UNIQUEMENT SI SES DEUX POINTEURS INTERNES (`tab` ET `data`) SONT `nil` : `(nil, nil)`**.

Que se passe-t-il si vous rangez un pointeur typé qui vaut `nil` (ex: `var p *ClientSMS = nil`) à l'intérieur d'une variable interface `var n Notificateur = p` ?

- Le champ `tab` de l'interface reçoit le descripteur du type `*ClientSMS` (`tab != nil` !).

- Le champ `data` de l'interface reçoit `nil`.
- L'interface vaut donc en mémoire **`(*ClientSMS, nil)`** → **La condition `n == nil` renvoie `false` !**

```Go
func piegeInterfaceNil() {
	var ptrSMS *ClientSMS = nil // Pointeur concret nil

	var notif Notificateur = ptrSMS // Copie dans l'interface : tab = *ClientSMS, data = nil

	if notif != nil {
		// ON ENTRE ICI CAR notif N'EST PAS (nil, nil) !
		fmt.Println("Piège : notif != nil est VRAI alors que le pointeur dedans est nil !")
	}
}
```

Comment éviter ce bug ? Ne retournez jamais un pointeur typé `nil` dans une fonction dont le type de retour est une interface ; retournez explicitement le littéral `nil`.

---

3. L'Interface vide (any), *Type Assertion* et *Type Switch*

3.1 L'interface vide any (`interface{}`)

Une interface qui n'exige **aucune méthode** (`interface{}`) est satisfaite par **100 % des types existants en Go** (`int`, `string`, `struct`, pointeurs...). Depuis Go 1.18, le langage fournit l'alias pré-déclaré **`any`** (strictement identique à `interface{}`). C'est l'équivalent d'un `void*` en C, mais **avec sécurité de type** car l'interface conserve le type exact de la donnée dans son premier pointeur interne !

3.2 Retrouver le type concret : *Type Assertion* et *Type Switch*

Lorsqu'on reçoit une interface (`Notificateur` ou `any`), comment accéder aux champs spécifiques de la `struct` concrète sous-jacente ?

```Go
func inspecterValeur(x any) {
	// 1. Type Assertion sécurisée avec l'idiome "Comma Ok" : val, ok := x.(TypeCible)
	// (Attention : sans le ', ok', si x n'est pas une string, le programme crash avec un panic !)
	if str, ok := x.(string); ok {
```

```
                fmt.Printf("C'est une chaîne de %d octets : %s\n", len(str), str)
        }

        // 2. Le Type Switch : aiguillage propre selon le type dynamique réel
        switch v := x.(type) { // Le mot-clé 'type' est littéral ici !
        case int:
                fmt.Printf("Entier (x2 = %d)\n", v*2) // Ici, 'v' est automatiquement
typé en 'int' !
        case ClientEmail:
                fmt.Printf("Client SMTP sur le serveur : %s\n", v.ServeurSMTP)
        case nil:
                fmt.Println("L'interface est totalement vide (nil, nil)")
        default:
                fmt.Printf("Type non géré : %T\n", v)
        }
}
```

---

4. Organisation des Packages et Architecture Modulaire

Un projet Go professionnel ne tient pas dans un seul fichier `main.go`. Il est découpé en **packages** (dossiers).

4.1 Règles fondamentales des packages en Go

1. **1 Dossier = 1 Package :** Tous les fichiers `.go` situés dans un même répertoire doivent impérativement commencer par le même `package nom_du_dossier` (exception faite du point d'entrée `package main`).
2. **Visibilité par la casse (Première lettre) :** Go n'a ni `public`, ni `private`, ni `protected`, ni fichiers d'en-tête `.h`.
    - **Première lettre en MAJUSCULE** (`type Client struct`, `func Connecter()`, `ChampPublic`) → **Exporté (Public)** : accessible depuis les autres packages qui importent celui-ci.
    - **Première lettre en minuscule** (`type config struct`, `func valider()`, `champPrive`) → **Non exporté (Privé au package)** : accessible uniquement par les fichiers `.go` situés dans le même dossier.
3. **Interdiction stricte des dépendances circulaires :** Si le package `A` importe le package `B`, le package `B` ne peut en aucun cas importer `A` (directement ou indirectement). Le compilateur refuse immédiatement de compiler, ce qui force une architecture en arbre propre (DAG) et explique la vitesse de compilation de Go.

4.2 Le dossier spécial `internal/` (Verrouillage par le compilateur)

En entreprise, comment partager un package `auth` entre plusieurs sous-packages de votre application **tout en interdisant** à des projets externes d'importer ce package `auth` ? Go intègre une règle directement dans son compilateur pour tout dossier nommé **`internal`** :

- Un package situé dans `.../mon-module/internal/crypto` ne peut être importé **que** par le code situé à l'intérieur de `.../mon-module/`. Tout import extérieur est bloqué à la compilation !

4.3 Structure standard d'un projet Go d'entreprise

```
None
mon-projet/
├── go.mod                    # Définition du module (ex:
github.com/entreprise/mon-projet)
├── cmd/                      # Points d'entrée exécutables (package main très
fins)
│   ├── api-server/
│   │   └── main.go           # Parse la config, câble les dépendances et lance le
serveur
│   └── worker-cli/
│       └── main.go
├── internal/                 # Code métier privé de l'application (protégé par le
compilateur)
│   ├── facturation/          # package facturation
│   │   ├── service.go        # Définit ses propres petites interfaces de stockage
!
│   │   └── modele.go
│   └── stockage/             # package stockage (implémente Postgres/Redis)
│       └── postgres.go
└── pkg/                      # (Optionnel) Librairies génériques publiques
réutilisables ailleurs
```